📝 Data Storage Formats: Character vs. Binary (Notes)

---

1. How Character Format Stores a Number

The **Character Format** breaks a number down into its individual **digits** (characters). The data stored in the file is the **ASCII value** of each character, not the actual binary representation of the integer value.

Concept	Description
Storage Unit	Each digit is stored as a character (typically **1 byte**).
Example (Integer 2594)	Characters: '2', '5', '9', '4'
File Contents	The file stores the ASCII codes:
	'2' $\rightarrow$ ASCII 50 $\rightarrow$ **00110010**
	'5' $\rightarrow$ ASCII 53 $\rightarrow$ **00110101**
	'9' $\rightarrow$ ASCII 57 $\rightarrow$ **00111001**
	'4' $\rightarrow$ ASCII 52 $\rightarrow$ **00110100**
Key Takeaway	These bytes are **NOT** the integer's true binary value; they are the **ASCII codes of the digits**.

---

2. The Conversion Process (Read/Write)

The critical aspect of Character Format is the necessary **internal conversion** between the integer value and its ASCII character representation.

➡️ Case A: Writing an Integer as Characters (e.g., file << x;)

This involves converting the integer's value into a sequence of characters, and then storing the ASCII byte for each character.

1. **Integer to Characters:** The integer value (e.g., 2594) is first converted into the characters '2', '5', '9', '4'.
2. **Store ASCII:** The ASCII byte for each character is written to the file.

⬅️ Case B: Reading Characters and Converting Back to Integer (e.g., file >> x;)

The extraction operator (>>) reads characters from the file and performs the reverse conversion to rebuild the number.

1. **Read Characters:** The stream reads the sequence of ASCII bytes representing the characters (e.g., '2', '5', '9', '4').
2. **ASCII to Digit:** Each character's ASCII value is converted into its numeric digit.
    - **Formula:** digit = character - '0' (This uses the ASCII difference, e.g., '2' (50) - '0' (48) = 2).
3. **Rebuild Number:** The digits are mathematically combined to reconstruct the original integer.
    - Example: $(((2 \times 10 + 5) \times 10 + 9) \times 10 + 4) = 2594$.

---

3. Why Character Format Requires Conversion

The conversion is necessary because the data in the file is stored as **human-readable** ASCII characters, not the raw, computational **binary** representation of the number.

Aspect	Character Format
Writing	Needs **integer** $\rightarrow$ **characters** conversion.
Reading	Needs **characters** $\rightarrow$ **integer** conversion.
Pros	✅ **Human Readable** (Can be opened and viewed in a text editor).

Cons	❌ **Slower** (due to conversion overhead).
	❌ **Larger** (requires more space; e.g., 4 bytes for 2594 vs. typically 2 bytes for the integer).

---

4. Binary Format vs. Character Format

Feature	💾 **Binary Format (e.g., write/read)**	🔠 **Character Format (e.g., <</put())**
What is Stored	The **actual integer bytes** (raw memory content).	The **ASCII codes of the digits**.
Conversion Needed	❌ **No**	✔️ **Yes** (Integer $\leftrightarrow$ Character/ASCII)
Speed	🚀 **Very Fast** (Direct memory copy).	🐌 **Slow** (Requires conversion logic).
Size	**Small** (Stores the number using its inherent size, e.g., 2 or 4 bytes).	**Larger** (Stores each digit as 1 byte).
Readability	Not human-readable in a text editor.	**Human-readable** in a text editor.

Would you like me to elaborate on the **Binary Format** and how it stores the actual integer bytes?